Báo cáo thực hành

Sinh viên thực hiện: Lâm Quang Khôi

MSSV: 3122411100 - Lớp: 3122411100

Chủ đề thực hiện: Lab 3 + 4

Lab 3: Attention

Mục tiêu

- Mở rộng mô hình Seq2Seq thực hiện được ở chương 2 trước đó.
- Thử nghiệm cơ chế Attention và hình dung qua hình vẽ.

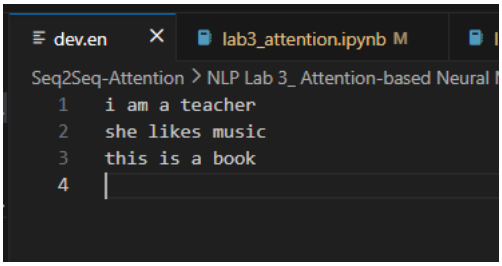
Phương pháp thực hiện

- Thực hiện các tasks:
  - Cài đặt cơ chế dot-product attention.
  - Điều chỉnh lại decoder để sử dụng biến context từ Attention.
  - Train mô hình dựa trên mô hình Seq2Seq và Attention vừa khởi tạo.
  - Đồ thị hóa qua đồ thị heatmap.

Quá trình thực hiện

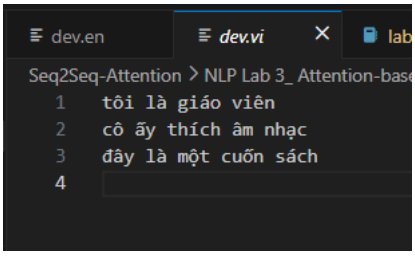
Dữ liệu sử dụng

- dev.en



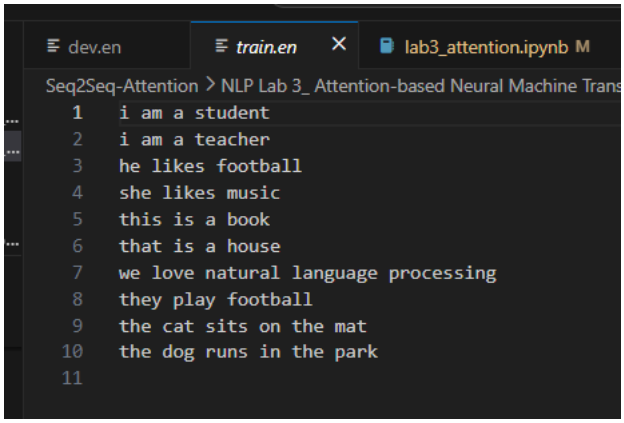
Pic 1. Data dev en.

- dev.vi



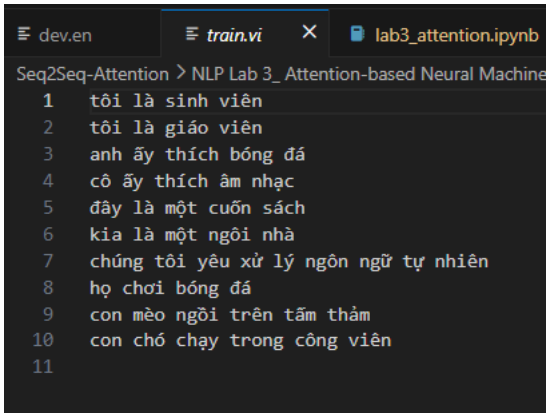
Pic 2. Data dev vi.

- train.en



Pic 3. Data train en

- train.vi



Pic 4. Data train vi.

Thực hành code

Trong phần thực hành cho sẵn không có các dòng code sẵn, cần phải thử cài đặt thử nghiệm thiết kế cơ chế Attention.

- Thiết kế cơ chế dot-product Attention.

```python
class DotAttention(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, query, keys, mask=None):
        """
        query: [batch, hidden]
        keys: [batch, src_len, hidden]
        mask: [batch, src_len], 1 for valid token, 0 for pad
        """
        scores = torch.bmm(keys, query.unsqueeze(2)).squeeze(2)  # [B, S]

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attn_weights = F.softmax(scores, dim=1)  # [B, S]
        context = torch.bmm(attn_weights.unsqueeze(1), keys).squeeze(1)  # [B, H]
        return context, attn_weights
```

Pic 5. Cài đặt hàm định nghĩa dotAttention.

- Chuẩn bị dữ liệu và hàm encoder-decoder.

```python
PAD_TOKEN = "<pad>"
SOS_TOKEN = "<sos>"
EOS_TOKEN = "<eos>"
UNK_TOKEN = "<unk>"
SPECIAL_TOKENS = [PAD_TOKEN, SOS_TOKEN, EOS_TOKEN, UNK_TOKEN]

def read_lines(path):
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip()]

def tokenize(sentence):
    return sentence.split()

def build_vocab(sentences, min_freq=1):
    counter = Counter()
    for s in sentences:
        counter.update(tokenize(s))

    itos = SPECIAL_TOKENS.copy()
    for token, freq in counter.items():
        if freq >= min_freq and token not in itos:
            itos.append(token)
    stoi = {tok: idx for idx, tok in enumerate(itos)}
    return stoi, itos

def encode(sentence, stoi):
    tokens = tokenize(sentence)
    ids = [stoi.get(SOS_TOKEN)]
    ids += [stoi.get(tok, stoi[UNK_TOKEN]) for tok in tokens]
    ids += [stoi.get(EOS_TOKEN)]
    return ids
```

Pic 6. Các hàm tiền xử lý dữ liệu thiết kế từ lab 1.

- Lớp Translation dataset.

```python
class TranslationDataset(Dataset):
    def __init__(self, src_lines, tgt_lines, src_stoi, tgt_stoi):
        assert len(src_lines) == len(tgt_lines), "Source/target size mismatch"
        self.pairs = list(zip(src_lines, tgt_lines))
        self.src_stoi = src_stoi
        self.tgt_stoi = tgt_stoi

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        src, tgt = self.pairs[idx]
        src_ids = torch.tensor(encode(src, self.src_stoi), dtype=torch.long)
        tgt_ids = torch.tensor(encode(tgt, self.tgt_stoi), dtype=torch.long)
        return src_ids, tgt_ids

def make_collate_fn(src_pad_id, tgt_pad_id):
    def collate(batch):
        src_batch, tgt_batch = zip(*batch)
        src_batch = nn.utils.rnn.pad_sequence(src_batch, batch_first=True, padding_value=src_pad_id)
        tgt_batch = nn.utils.rnn.pad_sequence(tgt_batch, batch_first=True, padding_value=tgt_pad_id)
        src_mask = (src_batch != src_pad_id).long()
        return src_batch, tgt_batch, src_mask
    return collate
```

Pic 7. Khởi tạo hàm TranslationDataset để tự động gom nhóm dữ liệu và thêm padding.

- Xây dựng mô hình Seq2Seq kết hợp với cơ chế Attention.

```python
class Encoder(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, pad_id):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.gru = nn.GRU(emb_dim, hidden_dim, batch_first=True)

    def forward(self, src):
        emb = self.embedding(src)                       # [B, S, E]
        outputs, hidden = self.gru(emb)                 # outputs: [B, S, H], hidden: [1, B, H]
        return outputs, hidden

class AttnDecoder(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, pad_id):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.attn = DotAttention()
        self.gru = nn.GRU(emb_dim + hidden_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim * 2, vocab_size)

    def forward(self, input_token, hidden, encoder_outputs, src_mask):
        # input_token: [B], hidden: [1, B, H]
        emb = self.embedding(input_token).unsqueeze(1)   # [B, 1, E]
        query = hidden[-1]                               # [B, H]

        context, attn_weights = self.attn(query, encoder_outputs, src_mask)
        rnn_in = torch.cat([emb, context.unsqueeze(1)], dim=-1)  # [B, 1, E+H]
        out, hidden = self.gru(rnn_in, hidden)           # out: [B, 1, H]

        logits = self.proj(torch.cat([out.squeeze(1), context], dim=-1))  # [B, V]
        return logits, hidden, attn_weights
```

Pic 8. Encoder và Decoder có cơ chế Attention (Lấy nhập liệu theo format từ tiền xử lý gom các batch lại).

```python
class Seq2SeqAttention(nn.Module):
    def __init__(self, encoder, decoder, tgt_sos_id):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.tgt_sos_id = tgt_sos_id

    def forward(self, src, tgt, src_mask, teacher_forcing_ratio=0.5):
        # src: [B, S], tgt: [B, T]
        batch_size, tgt_len = tgt.size()
        vocab_size = self.decoder.proj.out_features

        encoder_outputs, hidden = self.encoder(src)
        outputs = torch.zeros(batch_size, tgt_len - 1, vocab_size, device=src.device)
        all_attn = []

        input_token = tgt[:, 0]  # <sos>
        for t in range(1, tgt_len):
            logits, hidden, attn_weights = self.decoder(
                input_token, hidden, encoder_outputs, src_mask
            )
            outputs[:, t - 1] = logits
            all_attn.append(attn_weights)

            teacher_force = random.random() < teacher_forcing_ratio
            top1 = logits.argmax(dim=-1)
            input_token = tgt[:, t] if teacher_force else top1

        # [B, T-1, S]
        all_attn = torch.stack(all_attn, dim=1) if all_attn else None
        return outputs, all_attn
```

Pic 9. Mô hình Seq2Seq nhận Teacher Forcing là 0.5.

- Huẩn luyện mô hình.

```python
def compute_loss(logits, targets, pad_id):
    # logits: [B, T-1, V], targets: [B, T]
    logits = logits.reshape(-1, logits.size(-1))
    targets = targets[:, 1:].reshape(-1)
    return F.cross_entropy(logits, targets, ignore_index=pad_id)
```

Pic 10. Hàm Loss để mô hình chạy theo target.

```python
def run_epoch(model, loader, optimizer=None):
    is_train = optimizer is not None
    model.train() if is_train else model.eval()

    total_loss = 0.0
    for src, tgt, src_mask in loader:
        src = src.to(device)
        tgt = tgt.to(device)
        src_mask = src_mask.to(device)

        with torch.set_grad_enabled(is_train):
            logits, _ = model(
                src, tgt, src_mask,
                teacher_forcing_ratio=0.5 if is_train else 0.0
            )
            loss = compute_loss(logits, tgt, tgt_pad_id)

            if is_train:
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()

        total_loss += loss.item()
    return total_loss / max(1, len(loader))

emb_dim = 128
hidden_dim = 256

encoder = Encoder(len(src_itos), emb_dim, hidden_dim, src_pad_id).to(device)
decoder = AttnDecoder(len(tgt_itos), emb_dim, hidden_dim, tgt_pad_id).to(device)
model = Seq2SeqAttention(encoder, decoder, tgt_sos_id).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Pic 11. Khởi tạo hàm chạy huấn luyện theo từng epoch và xác định kích cỡ vector nhúng, vector ẩn nhúng, siêu tham số tối ưu hàm Adam.

```python
num_epochs = 5
for epoch in range(1, num_epochs + 1):
    train_loss = run_epoch(model, train_loader, optimizer=optimizer)
    dev_loss = run_epoch(model, dev_loader, optimizer=None)
    print(f"Epoch {epoch:02d} | train loss: {train_loss:.4f} | dev loss: {dev_loss:.4f}")
```

Pic 12. Thử nghiệm huấn luyện chạy 5 epoch (Có thể chỉnh sửa để xem visual Attention).

- Visualize từ có ngữ nghĩa cần tập trung của câu.

```python
# Simple attention visualization: focus on content words only.
sample_idx = 0
src_sentence = dev_en[sample_idx]
ref_sentence = dev_vi[sample_idx]

src_tokens = [SOS_TOKEN] + tokenize(src_sentence) + [EOS_TOKEN]
src_ids = [src_stoi.get(tok, src_stoi[UNK_TOKEN]) for tok in src_tokens]
src = torch.tensor(src_ids, dtype=torch.long, device=device).unsqueeze(0)
src_mask = (src != src_stoi[PAD_TOKEN]).long()

model.eval()
with torch.no_grad():
    enc_out, hidden = model.encoder(src)
    inp = torch.tensor([tgt_sos_id], dtype=torch.long, device=device)
    pred_tokens, attn_rows = [], []
    for _ in range(60):
        logits, hidden, attn_w = model.decoder(inp, hidden, enc_out, src_mask)
        next_id = int(logits.argmax(dim=-1).item())
        if next_id == tgt_eos_id:
            break
        pred_tokens.append(tgt_itos[next_id])
        attn_rows.append(attn_w.squeeze(0).cpu())
        inp = torch.tensor([next_id], dtype=torch.long, device=device)

print("Source:", src_sentence)
print("Reference:", ref_sentence)
print("Prediction:", " ".join(pred_tokens))
```

Pic 13. Khởi tạo visualize từ data gốc với data refrence để kiểm tra từ prediction tập trung

```python
if attn_rows:
    attn = torch.stack(attn_rows).numpy()  # [tgt_len, src_len]
    keep_cols = [i for i, tok in enumerate(src_tokens) if tok not in (SOS_TOKEN, EOS_TOKEN, PAD_TOKEN)]
    attn = attn[:, keep_cols]
    src_plot_tokens = [src_tokens[i] for i in keep_cols]

    plt.figure(figsize=(max(6, 0.6 * len(src_plot_tokens)), max(3, 0.5 * len(pred_tokens))))
    plt.imshow(attn, aspect="auto", cmap="viridis")
    plt.xticks(range(len(src_plot_tokens)), src_plot_tokens, rotation=45, ha="right")
    plt.yticks(range(len(pred_tokens)), pred_tokens)
    plt.xlabel("Source content words")
    plt.ylabel("Predicted words")
    plt.title("Attention Heatmap")
    plt.colorbar(label="Attention weight")
    plt.tight_layout()
    plt.show()
else:
    print("Model did not generate tokens for this sample.")
```

Pic 14. Thực hiện vẽ bằng Mathplotlib.

Kết quả thực hiện

- Quá trình thực hiện training.

```
Epoch 01 | train loss: 3.7868 | dev loss: 3.6725
Epoch 02 | train loss: 3.6301 | dev loss: 3.4549
Epoch 03 | train loss: 3.4397 | dev loss: 3.2563
Epoch 04 | train loss: 3.3009 | dev loss: 3.0658
Epoch 05 | train loss: 3.1337 | dev loss: 2.8936
Epoch 06 | train loss: 2.9516 | dev loss: 2.7230
Epoch 07 | train loss: 2.7466 | dev loss: 2.5228
Epoch 08 | train loss: 2.6249 | dev loss: 2.3626
Epoch 09 | train loss: 2.3182 | dev loss: 2.2646
Epoch 10 | train loss: 2.2139 | dev loss: 2.1175
Epoch 11 | train loss: 2.0736 | dev loss: 1.9552
Epoch 12 | train loss: 1.8297 | dev loss: 1.8413
Epoch 13 | train loss: 1.7101 | dev loss: 1.5373
Epoch 14 | train loss: 1.5569 | dev loss: 1.3850
Epoch 15 | train loss: 1.3607 | dev loss: 1.2240
Epoch 16 | train loss: 1.2187 | dev loss: 1.0828
Epoch 17 | train loss: 1.0766 | dev loss: 0.9381
Epoch 18 | train loss: 0.9512 | dev loss: 0.8143
Epoch 19 | train loss: 0.8581 | dev loss: 0.7130
Epoch 20 | train loss: 0.7562 | dev loss: 0.6232
Epoch 21 | train loss: 0.6677 | dev loss: 0.5380
Epoch 22 | train loss: 0.5891 | dev loss: 0.4577
Epoch 23 | train loss: 0.5160 | dev loss: 0.3895
Epoch 24 | train loss: 0.4509 | dev loss: 0.3317
Epoch 25 | train loss: 0.3921 | dev loss: 0.2831
...
Epoch 27 | train loss: 0.2933 | dev loss: 0.2001
Epoch 28 | train loss: 0.2510 | dev loss: 0.1683
Epoch 29 | train loss: 0.2154 | dev loss: 0.1447
Epoch 30 | train loss: 0.1862 | dev loss: 0.1254
```

Pic 15. Quá trình huấn luyện giảm hàm Loss.

- Kết quả sơ đồ visual Attention.

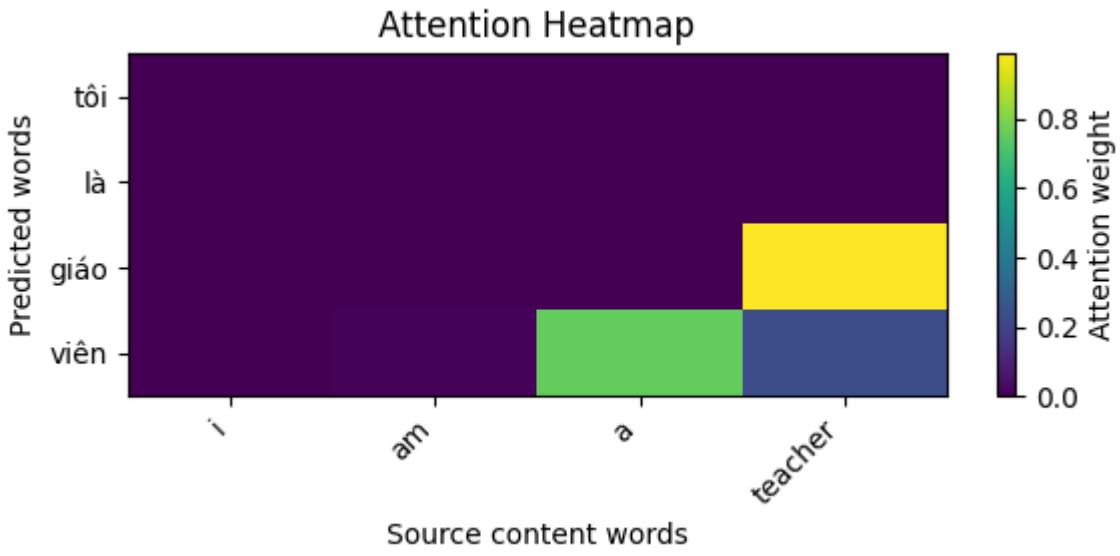


Figure 1. Kết quả sơ đồ heatmap.

→ Kết luận: Khi dự đoán từ "giáo", mô hình đã dồn gần như toàn bộ sự chú ý (trọng số ≈1.0, màu vàng sáng) vào từ gốc là "teacher". Điều này chứng tỏ cơ chế Attention đã hoạt động chính xác ở bước này, tạo ra được liên kết ngữ nghĩa đúng đắn. Sau đó, để dịch trọn vẹn khái niệm "a teacher" thành "giáo viên", khi sinh ra từ "viên", mô hình đã chia sẻ sự chú ý vào cả "a" (trọng số cao hơn, màu xanh lục) và một phần vào "teacher" (màu xanh thẫm). Đây là một hành vi hợp lý khi xử lý các từ ghép hoặc cụm từ tương đương giữa hai ngôn ngữ.

Giá trị rút ra

- Sự cải tiến so với Seq2Seq truyền thống: Cơ chế Attention giải quyết hiệu quả vấn đề bottleneck của kiến trúc Encoder-Decoder cổ điển bằng cách cho phép Decoder truy cập trực tiếp vào toàn bộ trạng thái ẩn của Encoder thay vì chỉ phụ thuộc vào vector ngữ cảnh duy nhất.
- Khả năng giải thích của mô hình: Thông qua việc tính toán điểm số tương đồng (scores) bằng phép nhân vô hướng (torch.bmm), mô hình không chỉ đưa ra kết quả dịch mà còn cung cấp khả năng giải thích (interpretability) thông qua trọng số Attention, giúp người học hiểu được từ nào trong câu gốc đóng góp vào việc tạo ra từ trong câu đích.
- Kỹ thuật xử lý Masking: Việc cài đặt hàm masked_fill với giá trị vô cùng (1e-9) trước khi tính Softmax là bước quan trọng để loại bỏ ảnh hưởng của các token đệm (<pad>), đảm bảo mô hình chỉ tập trung vào các thông tin hữu ích.

Lab 4: Mini Transformer NMT

Mục tiêu

- Khởi tạo một mô hình mini Transformer encoder-decoder cho mạng máy học dịch thuật.
- Hiểu được cách áp dụng mô hình Transformer trên bộ data nhỏ.

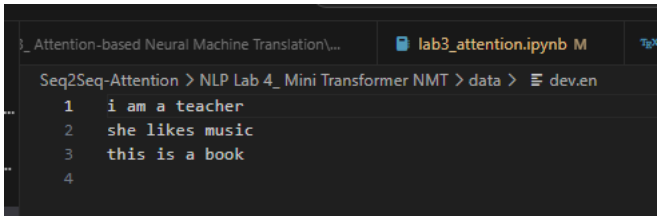
Phương pháp thực hiện

- Thực hiện các task:
    - Xây dựng hàm Vocabulary và encoder câu.
    - Xây dựng bộ encoding vị trí.
    - Dùng mô hình Transformer từ thư viện Pytorch.
    - Huấn luyện mô hình.
    - Xây dựng bộ greedy decoding.

Quá trình thực hiện

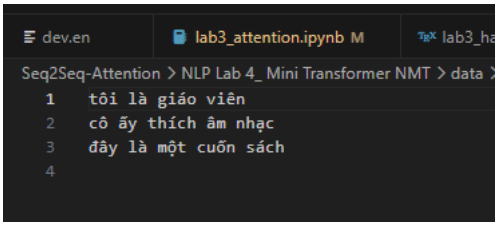
Dữ liệu sử dụng

- dev.en



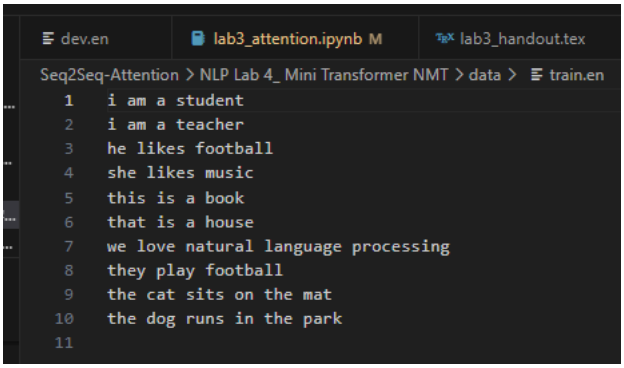
Pic 16. Data dev en.

- dev.vi



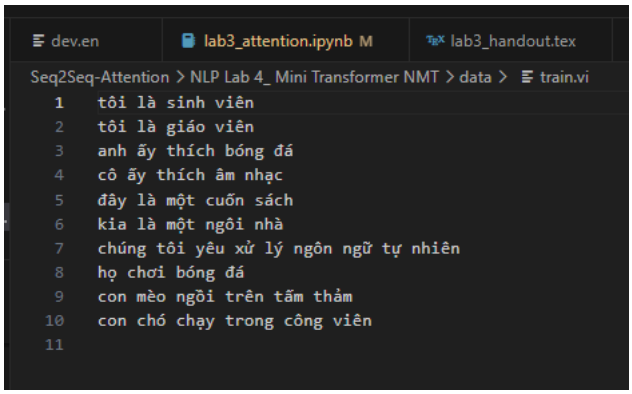
Pic 17. Data dev vi.

- train.en



Pic 18. Data train en.

- train.vi



Pic 19. Data train vi.

Thực hành code

- Xây dựng hàm Vocabulary và encode cả câu.

```python
PAD_TOKEN = "<pad>"
BOS_TOKEN = "<bos>"
EOS_TOKEN = "<eos>"
UNK_TOKEN = "<unk>"
SPECIAL_TOKENS = [PAD_TOKEN, BOS_TOKEN, EOS_TOKEN, UNK_TOKEN]

def tokenize(text: str):
    return text.strip().lower().split()

def read_parallel(src_path: Path, tgt_path: Path, max_samples: int | None = None):
    with src_path.open("r", encoding="utf-8") as fs, tgt_path.open("r", encoding="utf-8") as ft:
        src_lines = [line.strip() for line in fs if line.strip()]
        tgt_lines = [line.strip() for line in ft if line.strip()]

    n = min(len(src_lines), len(tgt_lines))
    pairs = list(zip(src_lines[:n], tgt_lines[:n]))
    if max_samples is not None:
        pairs = pairs[:max_samples]
    return pairs
```

Pic 20. Khởi tạo tiền xử lý dữ liệu.

```python
class Vocab:
    def __init__(self, tokens_list, min_freq=1):
        counter = Counter()
        for tokens in tokens_list:
            counter.update(tokens)

        self.itos = list(SPECIAL_TOKENS)
        for tok, freq in counter.items():
            if freq >= min_freq and tok not in self.itos:
                self.itos.append(tok)

        self.stoi = {tok: i for i, tok in enumerate(self.itos)}
        self.pad_id = self.stoi[PAD_TOKEN]
        self.bos_id = self.stoi[BOS_TOKEN]
        self.eos_id = self.stoi[EOS_TOKEN]
        self.unk_id = self.stoi[UNK_TOKEN]

    def encode(self, tokens):
        return [self.stoi.get(t, self.unk_id) for t in tokens]

    def decode(self, ids, skip_special=True):
        toks = []
        for idx in ids:
            tok = self.itos[idx]
            if skip_special and tok in SPECIAL_TOKENS:
                continue
            toks.append(tok)
        return toks

    def __len__(self):
        return len(self.itos)
```

Pic 21. Xây dựng hàm Vocab.

- Xây dựng bộ lưu dataset translation và định nghĩa kích cỡ batch.

```python
class TranslationDataset(Dataset):
    def __init__(self, pairs, src_vocab: Vocab, tgt_vocab: Vocab):
        self.examples = []
        for src_text, tgt_text in pairs:
            src_ids = [src_vocab.bos_id] + src_vocab.encode(tokenize(src_text)) + [src_vocab.eos_id]
            tgt_ids = [tgt_vocab.bos_id] + tgt_vocab.encode(tokenize(tgt_text)) + [tgt_vocab.eos_id]
            self.examples.append((torch.tensor(src_ids), torch.tensor(tgt_ids)))

    def __len__(self):
        return len(self.examples)

    def __getitem__(self, idx):
        return self.examples[idx]

def collate_batch(batch, pad_id_src, pad_id_tgt):
    src_list, tgt_list = zip(*batch)
    src_batch = pad_sequence(src_list, padding_value=pad_id_src)
    tgt_batch = pad_sequence(tgt_list, padding_value=pad_id_tgt)
    return src_batch, tgt_batch

train_dataset = TranslationDataset(train_pairs, src_vocab, tgt_vocab)
dev_dataset = TranslationDataset(dev_pairs, src_vocab, tgt_vocab)

BATCH_SIZE = 64

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    collate_fn=lambda b: collate_batch(b, src_vocab.pad_id, tgt_vocab.pad_id),
)
```

Pic 22. Khởi tạo hàm TranslationDataset.

- Tạo hàm mã hóa vị trí.

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        position = torch.arange(0, max_len).unsqueeze(1).float()
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(1)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)
```

Pic 23. Khởi tạo hàm mã hóa vị trí.

- Khởi tạo hàm mini Transformer.

```python
class MiniTransformerNMT(nn.Module):
    def __init__(
        self,
        src_vocab_size: int,
        tgt_vocab_size: int,
        d_model: int = 256,
        nhead: int = 8,
        num_encoder_layers: int = 3,
        num_decoder_layers: int = 3,
        dim_feedforward: int = 512,
        dropout: float = 0.1,
    ):
        super().__init__()
        self.d_model = d_model

        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model, dropout=dropout)

        self.transformer = nn.Transformer(
            d_model=d_model,
            nhead=nhead,
            num_encoder_layers=num_encoder_layers,
            num_decoder_layers=num_decoder_layers,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=False,
        )
        self.generator = nn.Linear(d_model, tgt_vocab_size)
```

Pic 24. Xác định kích thước và hàm định nghĩa.

```python
def _generate_square_subsequent_mask(self, sz: int, device):
    return torch.triu(torch.ones((sz, sz), dtype=torch.bool, device=device), diagonal=1)

def forward(self, src, tgt_in, src_pad_mask, tgt_pad_mask):
    src_emb = self.pos_encoder(self.src_embedding(src) * math.sqrt(self.d_model))
    tgt_emb = self.pos_encoder(self.tgt_embedding(tgt_in) * math.sqrt(self.d_model))

    tgt_mask = self._generate_square_subsequent_mask(tgt_in.size(0), tgt_in.device)

    memory = self.transformer.encoder(
        src_emb,
        src_key_padding_mask=src_pad_mask,
    )
    out = self.transformer.decoder(
        tgt_emb,
        memory,
        tgt_mask=tgt_mask,
        tgt_key_padding_mask=tgt_pad_mask,
        memory_key_padding_mask=src_pad_mask,
    )
    logits = self.generator(out)
    return logits
```

Pic 25. Gán giá trị mask trong cơ chế Transformer.

→ Trong cơ chế gốc của Transformer: Giá trị mask được gán để tránh mô hình có toàn bộ giá trị bao quát của dữ liệu để đưa ra dự đoán.

- Huấn luyện mô hình.

```python
def run_epoch(model, loader, optimizer=None):
    is_train = optimizer is not None
    model.train() if is_train else model.eval()

    total_loss = 0.0
    total_tokens = 0

    with torch.set_grad_enabled(is_train):
        for src, tgt in loader:
            src = src.to(device)
            tgt = tgt.to(device)

            tgt_in = tgt[:-1, :]
            tgt_out = tgt[1:, :]

            src_pad_mask = (src.transpose(0, 1) == src_vocab.pad_id)
            tgt_pad_mask = (tgt_in.transpose(0, 1) == tgt_vocab.pad_id)

            logits = model(src, tgt_in, src_pad_mask, tgt_pad_mask)
            loss = criterion(logits.reshape(-1, logits.size(-1)), tgt_out.reshape(-1))

            if is_train:
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()

            non_pad = (tgt_out != tgt_vocab.pad_id).sum().item()
            total_loss += loss.item() * non_pad
            total_tokens += non_pad

    return total_loss / max(total_tokens, 1)

NUM_EPOCHS = 30
for epoch in range(1, NUM_EPOCHS + 1):
    train_loss = run_epoch(model, train_loader, optimizer=optimizer)
    val_loss = run_epoch(model, dev_loader, optimizer=None)
    print(f"Epoch {epoch:02d} | train_loss={train_loss:.4f} | val_loss={val_loss:.4f}")
```

Pic 26. Hàm huấn luyện mô hình theo epoch.

- Greedy Decoding.

```python
def greedy_decode(model, src_ids, max_len=50):
    model.eval()
    src = torch.tensor(src_ids, dtype=torch.long, device=device).unsqueeze(1)
    src_pad_mask = (src.transpose(0, 1) == src_vocab.pad_id)

    generated = [tgt_vocab.bos_id]
    with torch.no_grad():
        for _ in range(max_len):
            tgt_in = torch.tensor(generated, dtype=torch.long, device=device).unsqueeze(1)
            tgt_pad_mask = (tgt_in.transpose(0, 1) == tgt_vocab.pad_id)

            logits = model(src, tgt_in, src_pad_mask, tgt_pad_mask)
            next_id = logits[-1, 0].argmax(dim=-1).item()
            generated.append(next_id)

            if next_id == tgt_vocab.eos_id:
                break

    return generated
```

Pic 27. Hàm Greedy decoding để lấy decode theo số có softmax cao nhất.

- Dịch thử nghiệm.

```python
def translate_sentence(sentence: str):
    src_ids = [src_vocab.bos_id] + src_vocab.encode(tokenize(sentence)) + [src_vocab.eos_id]
    pred_ids = greedy_decode(model, src_ids)
    pred_tokens = tgt_vocab.decode(pred_ids, skip_special=True)
    return " ".join(pred_tokens)

for i in range(min(5, len(dev_pairs))):
    src_text, tgt_text = dev_pairs[i]
    pred_text = translate_sentence(src_text)
    print(f"EN  : {src_text}")
    print(f"VI* : {pred_text}")
    print(f"VI  : {tgt_text}")
    print("-" * 80)
```

Pic 28. Hàm dịch thử nghiệm dựa vào hàm greedy decoding đã có.

Kết quả thực hiện

- Quá trình huấn luyện giảm hàm Loss.

```
Epoch 01 | train_loss=4.0611 | val_loss=3.2936
Epoch 02 | train_loss=3.6028 | val_loss=3.0433
Epoch 03 | train_loss=3.4175 | val_loss=2.8771
Epoch 04 | train_loss=3.2620 | val_loss=2.7226
Epoch 05 | train_loss=3.1125 | val_loss=2.5740
Epoch 06 | train_loss=2.9400 | val_loss=2.4052
Epoch 07 | train_loss=2.8240 | val_loss=2.2445
Epoch 08 | train_loss=2.6474 | val_loss=2.0944
Epoch 09 | train_loss=2.5783 | val_loss=1.9636
Epoch 10 | train_loss=2.3883 | val_loss=1.8398
Epoch 11 | train_loss=2.3345 | val_loss=1.7334
Epoch 12 | train_loss=2.2498 | val_loss=1.6281
Epoch 13 | train_loss=2.0780 | val_loss=1.5387
Epoch 14 | train_loss=1.9956 | val_loss=1.4629
Epoch 15 | train_loss=1.8776 | val_loss=1.3944
Epoch 16 | train_loss=1.8558 | val_loss=1.3236
Epoch 17 | train_loss=1.7348 | val_loss=1.2533
Epoch 18 | train_loss=1.6791 | val_loss=1.1822
Epoch 19 | train_loss=1.5933 | val_loss=1.0961
Epoch 20 | train_loss=1.4988 | val_loss=1.0200
Epoch 21 | train_loss=1.5208 | val_loss=0.9505
Epoch 22 | train_loss=1.3147 | val_loss=0.8871
Epoch 23 | train_loss=1.3087 | val_loss=0.8143
Epoch 24 | train_loss=1.2427 | val_loss=0.7466
Epoch 25 | train_loss=1.1779 | val_loss=0.6862
...
Epoch 27 | train_loss=0.9937 | val_loss=0.5765
Epoch 28 | train_loss=1.0456 | val_loss=0.5286
Epoch 29 | train_loss=0.9114 | val_loss=0.4852
Epoch 30 | train_loss=0.8859 | val_loss=0.4478
```

Pic 29. Quá trình giảm hàm Loss khi huấn luyện.

- Kết quả dịch thuật.

```
EN  : i am a teacher
VI* : tôi là giáo viên
VI  : tôi là giáo viên
--------------------------------------------------------------------------------
EN  : she likes music
VI* : cô ấy thích âm nhạc
VI  : cô ấy thích âm nhạc
--------------------------------------------------------------------------------
EN  : this is a book
VI* : đây là một cuốn sách
VI  : đây là một cuốn sách
--------------------------------------------------------------------------------
```

Pic 30. Kết quả dịch thuật

→ Đánh giá: Dẫu do bộ dữ liệu khá đơn giản nên kết quả rất dễ bị overfitting, nhưng kết quả đã cho thấy hiệu quả của việc áp dụng Transformer nhỏ.

Giá trị rút ra

- Ưu thế của kiến trúc Transformer: Việc thay thế các lớp tuần tự (RNN) bằng cơ chế Self-Attention và Positional Encoding giúp mô hình nắm bắt các mối quan hệ xa trong câu (long-range dependencies) tốt hơn mà không bị ảnh hưởng bởi vấn đề triệt tiêu đạo hàm.
- Tầm quan trọng của Positional Encoding: Do Transformer không có tính tuần tự tự nhiên, việc cộng thêm vector vị trí (sin, cos) là bắt buộc để mô hình nhận biết được thứ tự trước sau của các từ, một yếu tố sống còn trong ngôn ngữ.
- Cơ chế Masking trong Transformer: Lab 4 cho thấy sự phức tạp hơn trong việc quản lý Mask, đặc biệt là square_subsequent_mask ở phía Decoder để đảm bảo tính nhân quả (causality) - tức là mô hình không được phép "nhìn thấy" các từ trong tương lai khi đang trong quá trình huấn luyện.
- Thách thức với dữ liệu nhỏ: Transformer là mô hình cần rất nhiều dữ liệu. Khi áp dụng trên bộ data nhỏ, mô hình dễ gặp hiện tượng overfitting hoặc lỗi lặp từ, đòi hỏi các kỹ thuật điều chỉnh như Dropout (0.1) và Label Smoothing hoặc Beam Search để cải thiện chất lượng đầu ra.